

TensorForge 2.0 Phase 2 — Technical Report

Oct 6, 2026 · @osal_geesara

Team Nexio, University of Moratuwa: H.K.Osal Geesara, Sahas Samudhitha, Chamalka Heshani, Poojani Danulya, Rashmi Abeysekara

1. Summary

Our fine-tuned multilingual transformer routes RideEat support tickets to the right team with **85.3% category accuracy** on the 800-ticket validation set, against 60.4% for a TF-IDF baseline. It also predicts a secondary issue and urgency (urgent F1 0.969, 98.8% of urgent tickets caught), and gets all three outputs right on 83.4% of tickets.

The model is `xlm-roberta-base` with three output heads, trained by us; no LLM API is called. It runs on CPU as ONNX behind a FastAPI service that follows the OpenAPI contract, works offline in Docker, and passes 252 of 252 contract checks against the hosted HTTPS URL. API: <https://13-232-241-157.sslip.io> · demo: <https://13-232-241-157.sslip.io/demo/> · code: <https://github.com/osalhk/tensorforge>

2. Problem and data

From `channel`, `subject` and `text` alone we predict the `category` (11 classes), an optional `secondary_category` that must differ from it, and `is_urgent`. The `team` is never learned: it follows from the category through the specification's fixed table.

The data has 4,000 training and 800 validation tickets with no missing values or duplicates. It mixes English (35%), Singlish (25%), Sinhala (20%), Tamil (15%), Tanglish and mixed text; the `language` column is never sent to the API, so the model works without it. Classes are imbalanced six-fold (720 `payment_refund` vs 120 `spam_irrelevant`), and only 10% of tickets are urgent, but 75% of safety tickets are.

3. Model approach and design decisions

Classical models could not read Sinhala, Tamil and mixed-script tickets well, so we fine-tuned a multilingual transformer; the rules allow fine-tuned models but not LLM wrappers.

Decision	Why
Baseline first: TF-IDF + logistic regression	A reproducible reference to beat (0.604)

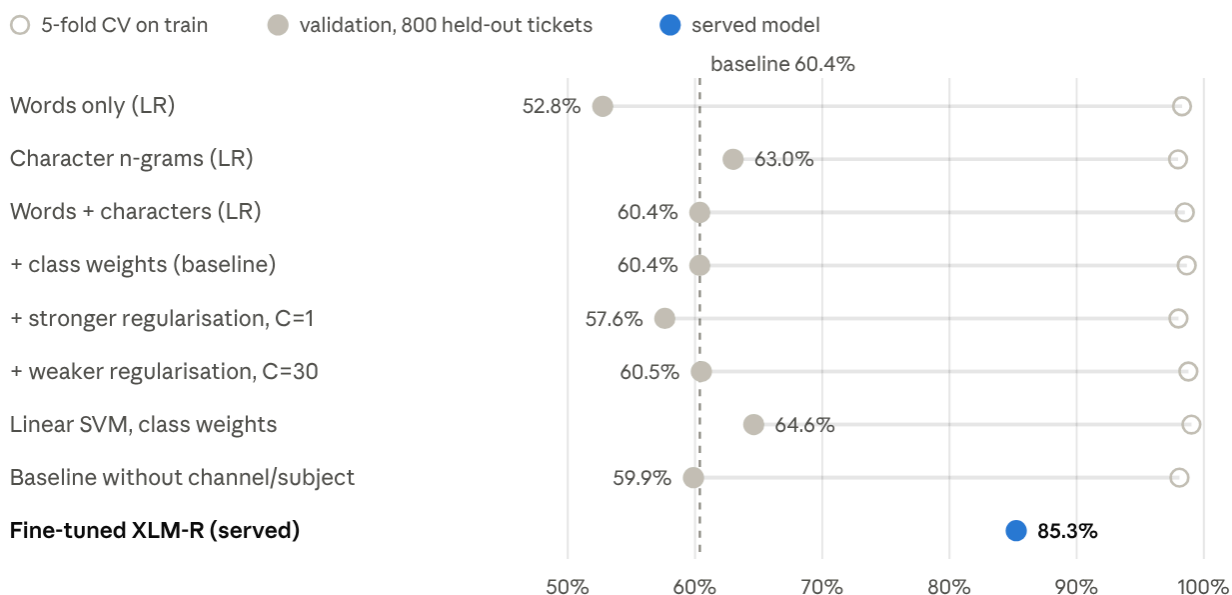
Decision	Why
xlm-roberta-base , one encoder, three heads	Covers Sinhala and Tamil; outputs share evidence in one pass
Class weights for rare classes	Safety, spam and urgent tickets are rare but costly to miss
Fixed rules after the model (team table, secondary $\neq$ primary, spam never urgent)	The spec fixes them; learning them could only add errors
fp32 ONNX on CPU	Fits 2 vCPU / 4 GB with no GPU or PyTorch
Temperature scaling; human review below 0.5 confidence	Honest confidence, which the spec scores

4. Training, validation and improvement

We improved the model in logged, reproducible steps, each scored on the 800 held-out validation tickets. Every run is in `reports/experiments.csv` , regenerated by `notebooks/04_experiments.py` with seed 42.

Fine-tuning lifted validation accuracy to 85%; no classical model passed 65%

Category accuracy per experiment, despite ~98% in cross-validation on train



`reports/experiments.csv` · 8 classical experiments + served model, validation set of 800 tickets

Cross-validation was a trap. Classical models scored ~98% in 5-fold CV on train but 53–65% on validation. Train holds near-paraphrases: a training ticket's nearest neighbour shares its category 82% of the time, a validation ticket's only 31%. We therefore compared

models on validation only. Among classical models, character n-grams beat words (63.0% vs 52.8%) and a linear SVM did best (64.6%).

Fine-tuning. AdamW at $2e-5$ ($3e-5$ was unstable), batch 16, 8 epochs, two seeds; loss = category + $0.5 \times$ secondary + $0.5 \times$ urgency. Accuracy reached 85% by epoch 5–6 for both seeds; we kept seed 42, epoch 6, and tuned the urgency threshold (0.45) for F1.

Serving. int8 quantisation cost 3–4 points, so we serve fp32. Temperature scaling ($T = 1.75$, stable at 1.72–1.77 across folds) cut calibration error from 0.106 to 0.025 without changing any prediction.

5. Evaluation results

The served model beats the baseline on every output (800 validation tickets, scored through the same code as `/predict`).

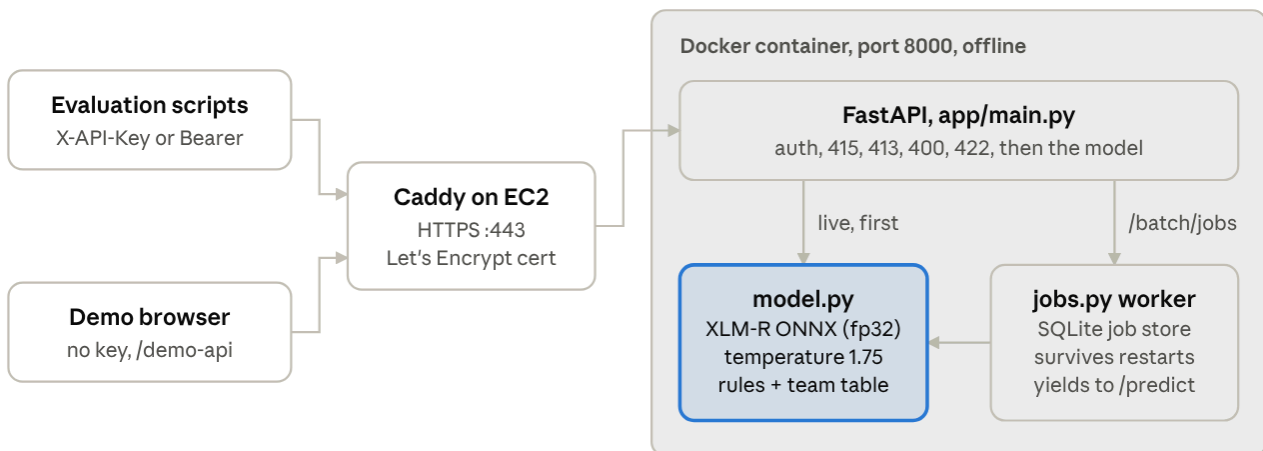
Metric	TF-IDF baseline	Fine-tuned XLM-R (served)
Category accuracy (= team accuracy)	0.604	0.853
Category macro-F1	0.631	0.851
Secondary category accuracy / macro-F1	0.953 / 0.668	0.979 / 0.877
Urgent precision / recall / F1	0.821 / 0.863 / 0.842	0.952 / 0.988 / 0.969
All three outputs correct	0.564	0.834
Expected calibration error	0.211	0.024
Consistency rules hold	yes	yes

F1 is highest for `safety_conduct` and `account_promo` (0.95) and lowest for `general_inquiry` (0.67). The largest confusion is vague refund requests sent to `general_inquiry` (25 of 144).

6. System architecture and API compliance

The service implements all seven endpoints of the OpenAPI contract exactly, and checks every request in the specified order before any model work.

Auth runs before the model; live requests run before job work



system architecture · hosted deployment

- **Auth and errors.** `X-API-Key` or `Bearer`, compared in constant time with `API_KEY`; unset key → 401, never open. Every error, including 404/405, uses the spec's error body, with `index` per failing batch item.
- **Async jobs.** Up to 5,000 tickets, validated before creation; SQLite state, paging, 429 when full, results kept 24 h; a restart ends a running job as `failed` / `interrupted`. Live requests run before job and demo work.
- **Demo.** The page never holds the key: it calls rate-limited, low-priority `/demo-api` routes running the same model.

7. Deployment, testing and robustness

Every item on the official checklist was verified by a test or a measured run.

Checklist item	Evidence
Schema-valid responses	64 pytest tests validate every response and error with <code>jsonschema</code>
Key required, <code>/health</code> public	401 on all 6 protected endpoints without a valid key
One command, no internet	Healthy in ~4 s with <code>--network none</code> , 2 CPUs, 4 GB; 1.55 GB RAM
Hosted and tested end-to-end	<code>scripts/smoke_test.py</code> : 252 / 252 checks on the HTTPS URL
Bad input never crashes	Clean 400/413/415/422 JSON errors, never a 5xx

The service runs on AWS EC2 (2 vCPU, 4 GB) behind Caddy with a Let's Encrypt certificate. The smoke test caught one real bug before submission (an oversized upload got a dropped connection instead of a 413), which we fixed.

8. Findings, limitations and next steps

Findings. A fine-tuned multilingual encoder was the decisive step, in every language. Honest held-out validation mattered more than any model tweak, because cross-validation on this training set overstates accuracy. Calibration was the cheapest real improvement.

Limitations. The checkpoint and urgency threshold were chosen on validation, so the hidden-test score may be slightly lower. Singlish is the weakest language (0.79) and vague refunds drift to `general_inquiry`. The synthetic Sinhala and Tamil text was not reviewed by native speakers.

Next steps. Retrain on train + validation, ensemble with the character-level model for Singlish, and add examples separating vague refunds from general questions.